

HATS: A Heterogeneous Agent Tool Substrate for Agentic Prefill

HATS Project

May 5, 2026

Abstract

Agentic AI systems increasingly combine large model inference with tool use: retrieval, structured parsing, schema validation, ranking, memory lookup, code-like transformations, cancellation, retry, and dependency scheduling. Meanwhile, much accelerator work is specializing the autoregressive decode path: one-token continuation, KV-cache reads, and even separate attention and feed forward execution. HATS takes the complementary position. It specializes the agentic prefill path: the construction, validation, retrieval, filtering, and ingestion of context before model continuation. In today’s systems, the dense tensor work is placed on GPUs, while much of this agentic prefill and re-prefill work remains on the CPU. This split is convenient, but it can create high-latency, high-power execution patterns: model decoding emits a tool request, host software wakes up, validates arguments, launches another kernel or IO operation, copies or maps results, builds continuation context, and then resumes GPU-side inference.

This paper proposes HATS, a Heterogeneous Agent Tool Substrate inspired by HSA principles but scoped to bounded local tools for agentic AI. HATS is not a proposal to replace the CPU with a GPU control processor. Instead, it is a compiler/runtime substrate that exposes coherent memory, user-mode queues, fine-grained signals, capability-scoped tool access, and a narrow set of hardware-friendly tool primitives. The initial target set includes retrieval, structured parsing, schema validation, context construction, tool-result ingestion, dependency scheduling, data movement, and sandboxed pure compute kernels.

The central claim is conditional and empirical: agentic AI does not justify moving arbitrary external tools into accelerator hardware, but it may justify moving frequent, memory-resident prefill-side tool primitives into a coherent GPU-side runtime. HATS therefore begins as a compiler and runtime project that can run on existing heterogeneous systems, measure bottlenecks, and identify which hardware primitives have credible performance, power, and area value.

1 Introduction

Modern AI accelerators are designed around dense numerical kernels. This has served neural network training and inference well: the central optimization problem is to move tensors through high-throughput matrix engines efficiently. Agentic AI changes the workload shape. A useful agent does not merely decode tokens; it plans, branches, retrieves memory, validates structured arguments, invokes tools, cancels stale work, merges results, ranks alternatives, builds fresh context, and resumes model execution.

This paper focuses on the prefill side of that loop. Conventional prefill converts an input prompt into model state, including a KV cache that can be used by subsequent decode. Agentic systems create prefill-like events repeatedly: initial prompts are prefetched once, but tool results, retrieved memories, ranked candidates, and repaired structured arguments must be ingested throughout the workflow. These re-prefill events are where HATS expects the strongest opportunity. Decode

should continue to be specialized aggressively, but HATS targets the repeated construction of useful context around decode.

This outer loop is not a small detail. Many agentic workflows spend substantial time outside the core matrix multiplication path. A common execution pattern is:

```
GPU decode emits a tool boundary
-> CPU observes tool request
-> CPU validates arguments
-> CPU launches retrieval or parsing work
-> GPU or CPU computes the tool result
-> CPU synchronizes and formats the result
-> CPU/GPU builds continuation context and KV state
-> GPU resumes decoding
```

Each transition may be reasonable in isolation. Together, the pattern can produce poor locality, frequent host wakeups, heavyweight synchronization, and unnecessary movement of data that was already close to GPU-resident model state. The power and latency cost is especially visible when tool calls are small, frequent, and dependent on fine-grained model outputs.

HATS asks whether the agentic outer loop should have a first-class heterogeneous execution substrate. The answer should not be assumed. General branching, OS policy, arbitrary network APIs, browsers, and shell tools remain natural CPU responsibilities. The narrower opportunity is different: many agentic systems repeatedly use bounded, local, memory-oriented primitives that are close to the model execution path. These include vector retrieval over resident indexes, key-value memory lookup, JSON or schema validation, constrained decoding checks, fan-out/fan-in scheduling, cancellation propagation, and small deterministic scoring or filtering functions.

HATS is inspired by HSA’s core philosophy: heterogeneous processors should cooperate through shared virtual memory, queues, and signals rather than through heavy host-mediated boundaries. HATS applies that philosophy to agentic tool workflows. The result is not a general-purpose operating system inside a GPU, but a bounded substrate for local tool primitives in the agentic prefill path.

2 Motivation

2.1 Agentic Workloads Are Not Pure Tensor Programs

Large language model inference is often presented as a sequence of decode steps. In agentic systems, those decode steps are embedded in a dynamic control program. The model may emit a structured tool request, the runtime may validate the request against a schema, retrieval may fetch relevant memory, and the result may alter the continuation path. These operations are fine-grained and dependency-heavy.

The mismatch with conventional GPU execution is not only control flow. It is also ownership. Tool state may live in CPU memory, vector indexes may live in GPU memory, parsed arguments may cross host-device boundaries, and scheduling metadata may be maintained by a runtime that has no cheap way to expose fine-grained futures to GPU work. As a result, the system pays for orchestration around the tensor kernels.

2.2 Agentic Prefill Is Repeated Work

The usual prefill/decode split treats prefill as the bulk parallel stage and decode as the latency-sensitive sequential stage. Many current accelerator efforts therefore specialize decode: minimize

per-token latency, optimize KV-cache reads, and split attention from feed-forward computation so each can be served by more specialized hardware.

HATS is aimed at a different, under-specialized part of the loop. Tool-using agents repeatedly create new context that must be validated, retrieved, filtered, ranked, transformed, and ingested before the model can continue. This is prefill-like work, even when it occurs in the middle of a conversation. A tool result may be short, but the system around it may touch resident memory indexes, schema state, ranking kernels, context buffers, and KV-cache update paths. HATS names this workload *agentic prefill*.

Agentic prefill has a better PPA shape than general decode control. It exposes parallelism over retrieved documents, candidate memories, structured fields, and context blocks. It is often close to model state and local indexes. It can benefit from coherent memory, cheap queues, and fine-grained signals without asking the accelerator to own high-level planning or arbitrary external tools.

2.3 The PPA Question

The performance, power, and area question is the core discipline for HATS. CPUs are excellent at irregular scalar control. A new accelerator feature is not justified simply because it can execute tool-related instructions. It is justified only if it saves enough latency or energy to compensate for its area and design complexity.

The strongest PPA case for HATS comes from avoiding unnecessary boundaries:

- avoiding CPU wakeups for small, frequent tool operations;
- avoiding host-device synchronization when prefill or re-prefill state is already GPU-resident;
- avoiding copies between model state, retrieval indexes, and tool buffers;
- keeping structured validation near constrained decoding and context ingestion;
- batching or fusing tool operations that would otherwise be tiny kernels;
- letting a low-area scheduler manage dependency metadata close to the queues it serves.

The weak case is equally important. HATS should not accelerate arbitrary browser control, network APIs, shell commands, or general-purpose planning logic. These operations have variable latency, security-heavy semantics, and often poor locality with model state. The CPU and operating system should remain responsible for them.

3 Design Principles

3.1 Specialize Prefill, Not General Decode

HATS should be understood as a prefill-side accelerator substrate. It is not a replacement for specialized decode engines, and it should not compete with hardware whose only job is to minimize one-token latency. Instead, HATS specializes the preparation of model continuation: retrieval, argument validation, result filtering, context assembly, placement, and dependency signals that make the next prefill or decode step cheaper to start.

This distinction keeps the architecture disciplined. The decode path remains thin and latency-oriented. The HATS path is wider and locality-oriented: it handles the repeated construction of context around tool boundaries, where there is enough parallelism and memory traffic to justify heterogeneous mechanisms.

3.2 Bounded Tools First

HATS limits its initial scope to local, bounded tool primitives:

1. **Retrieval:** vector search, key-value memory lookup, embedding cache access, and local index traversal.
2. **Structured parsing:** JSON parsing, schema validation, argument extraction, tokenization, and constrained decoding predicates.
3. **Context construction:** packing retrieved or tool-produced data into continuation buffers, updating prefill metadata, and preparing KV-cache construction paths.
4. **Graph scheduling:** futures, dependency tracking, fan-out/fan-in, retry, timeout, and cancellation.
5. **Data movement:** DMA and placement between model state, tool buffers, indexes, CPU memory, storage, and network-facing buffers.
6. **Sandboxed pure compute:** deterministic kernels for filtering, ranking, scoring, formatting, or lightweight transformations.

This scope is narrow enough to make a hardware conversation meaningful, but broad enough to cover common hot paths in agentic systems.

3.3 CPU as Policy, HATS as Local Mechanism

HATS does not eliminate CPU orchestration. Instead, it changes the boundary. The CPU remains responsible for high-level planning, OS policy, authentication, network security, external tool mediation, and arbitrary software execution. HATS handles frequent local mechanisms near model execution: dispatch, signals, local retrieval, validation, context construction, and bounded compute.

3.4 Compiler/Runtime Before Silicon

HATS should begin as a compiler and runtime project. The first version can lower agentic tool graphs onto existing CPUs, GPUs, and heterogeneous runtimes. That implementation provides the measurement basis for hardware proposals. If vector retrieval dominates, near-memory search engines may be justified. If parsing dominates, schema units may be justified. If synchronization dominates, hardware futures and queues may be justified.

4 HSA Lineage

HSA introduced a useful model for heterogeneous cooperation. Rather than treat the GPU as a device controlled exclusively by host driver calls, HSA emphasizes shared virtual memory, user-mode queues, and low-cost signals between heterogeneous agents. HATS borrows these ideas while changing the target workload.

HATS should avoid reviving older portable instruction set ideas directly. Instead, it can use modern compiler infrastructure. A plausible stack is an MLIR dialect for agentic tool graphs, lowering passes to GPU kernels, CPU tasks, SPIR-V-like compute targets, or sandboxed bytecode, and runtime packets that look like HSA queues and signals.

HSA Concept	HATS Adaptation	Agentic Role
Shared virtual memory	Coherent tool/model address space	Avoid copies and remapping
User-mode queues	Tool dispatch queues	Cheap small-tool launch
Signals	Futures and cancellation	Fine-grained dependencies
Heterogeneous agents	Tool engines and schedulers	Route work by locality
Memory regions	Capability-scoped buffers	Bound tool access

Table 1: HSA concepts adapted for HATS.

5 Programming Model

At the programming level, an agentic tool operation can be represented as a future-producing call with explicit capabilities and schemas:

```
future<Result> tool_call(
    capability cap,
    tool_id tool,
    schema_id input_schema,
    void* input,
    signal cancel
);
```

This abstraction is intentionally restrictive. The capability describes the memory regions and tool classes available to the call. The schema identifies the expected input structure. The cancellation signal lets speculative or stale agent branches stop consuming resources. The future becomes a dependency object that can wake a model continuation, another tool, or a CPU policy handler.

At a lower level, the runtime may represent a tool request as a packet:

```
struct hats_packet {
    uint32_t opcode; // retrieve, validate, rank, dma, sandbox, ...
    uint32_t capability_id; // bounds memory and tool access
    uint64_t input_ptr;
    uint64_t output_ptr;
    uint64_t schema_or_kernel;
    uint64_t completion_signal;
    uint64_t cancellation_signal;
    uint32_t flags;
    uint32_t priority;
};
```

The packet form matters because it is the bridge between compiler IR and heterogeneous execution. Different backends can consume the same logical packet: a CPU task runner, a GPU kernel, a retrieval engine, a parser unit, or a simulator for proposed hardware.

6 Compiler and Runtime Architecture

6.1 Agent Tool Graph IR

HATS begins with an Agent Tool Graph IR. This IR captures tool-producing model steps, dependencies, structured data contracts, and cancellation edges. Unlike a pure tensor IR, it must represent dynamic control and future-like values:

```

%args = hats.decode_until_tool %model_state
%ok = hats.validate %args schema(@SearchArgs)
%hits = hats.retrieve %index, %args.query if %ok
%rank = hats.sandbox @rerank(%hits, %args)
%ctx = hats.assemble_context %rank, %args
%kv = hats.prefill_context %model_state, %ctx
hats.resume_decode %model_state, %kv

```

The IR should be high-level enough to preserve agentic structure and low-level enough to expose placement decisions. The compiler can decide whether validation should run as CPU code, GPU code, a parser primitive, or fused logic near constrained decoding, and whether context construction should be batched with the next prefill stage.

6.2 Lowering

The lowering pipeline can proceed in stages:

1. **Graph normalization:** identify tool boundaries, schemas, dependency edges, cancellation scopes, and retry regions.
2. **Placement:** choose CPU, GPU, retrieval, parser, DMA, or sandbox targets based on data locality and profiling.
3. **Packetization:** lower tool operations to HATS packets and dependency signals.
4. **Fusion and batching:** group tiny operations when their dependencies and memory regions permit.
5. **Backend emission:** generate CPU tasks, GPU kernels, sandbox bytecode, or simulator instructions.

6.3 Runtime

The runtime maintains queues, capability tables, signal objects, and memory regions. Its job is not to make arbitrary distributed systems calls. Its job is to keep local tool work near the data it consumes and the continuation it unblocks.

The runtime should expose profiling hooks for:

- host wakeups avoided or incurred;
- queue latency and signal latency;
- bytes moved across CPU-GPU boundaries;
- tool operation size distribution;
- cancellation latency;
- occupancy of retrieval, parser, and sandbox engines;
- continuation latency from tool completion to usable prefill or KV state.

7 Candidate Hardware Primitives

HATS does not require all of the following primitives. They are hypotheses to evaluate through the compiler/runtime prototype.

7.1 Queue and Signal Fabric

A low-area queue and signal fabric may provide the highest return. Agentic tool work often has many small dependent tasks. If every task requires a host driver round trip, the system loses even when the computation itself is cheap. Hardware queues, futures, and cancellation signals can reduce latency and CPU wakeups.

7.2 Retrieval Blocks

Retrieval is a strong candidate for near-data acceleration. Embedding indexes, attention caches, and memory stores may already live in HBM or unified memory. Vector distance computation, top-k selection, and coarse filtering map naturally to parallel hardware. The PPA case is strongest when the index is resident and the result feeds directly into agentic prefill or GPU-side continuation.

7.3 Structured Parsing and Schema Units

Agentic systems frequently produce structured arguments. JSON validation, schema checks, token-level constraints, and format repair can become hot paths around decoding and context ingestion. Specialized parsers do not need to be large. Even modest support for finite-state validation, bracket matching, type checks, and field-level constraints may reduce CPU intervention and failed tool calls.

7.4 Sandboxed Microkernel Engine

Some tools are small deterministic functions: filter these hits, rank these items, transform this structure, compute a score, or normalize this result. A WASM-like or eBPF-like sandbox target could support these functions with explicit capabilities and bounded execution. The goal is not general software compatibility. The goal is safe, predictable local compute near the data.

7.5 Data Movement Engine

Many agent workflows are limited by placement rather than arithmetic. A HATS data movement engine would understand tool buffers, model continuations, prefill buffers, retrieval indexes, and CPU-visible memory regions. It would use capabilities to move only authorized regions and signals to wake dependent work.

8 Security and Isolation

Tool use is a security boundary. HATS must make this boundary explicit.

Capabilities should specify:

- which memory regions a tool can read or write;
- which queues it can submit to;
- which signals it can wait on or complete;

- which schemas or kernels it can invoke;
- whether it can trigger CPU-mediated external tools.

External tools should remain behind CPU and OS policy. HATS can prepare, validate, rank, or format requests, but network APIs, browser automation, shell commands, and filesystem side effects require explicit mediation. This boundary keeps the first HATS design tractable and prevents the architecture from becoming a confused general-purpose OS bypass.

9 Evaluation Plan

The first HATS prototype should answer whether the substrate is worth building. It should compare a baseline CPU-orchestrated agent runtime against a HATS-style runtime on existing hardware.

9.1 Benchmarks

Useful benchmarks include:

1. **Retrieve-prefill-resume:** model emits retrieval requests, local vector search runs, retrieved context is packed, and decode resumes from the resulting prefill or KV state.
2. **Schema-heavy tool use:** model emits structured arguments that must be validated and repaired before tool dispatch or context ingestion.
3. **Speculative fan-out:** multiple retrieval or ranking branches launch and stale branches are cancelled before re-prefill.
4. **Small-kernel pipeline:** many small deterministic transforms run between decode and re-prefill steps.
5. **Memory locality stress:** tool buffers, context buffers, and indexes vary across CPU memory, GPU memory, unified memory, and storage-backed regions.

9.2 Metrics

The evaluation should report:

- end-to-end task latency;
- tokens per joule for tool-using decode and re-prefill;
- time from tool result to usable continuation context or KV state;
- host wakeups per completed tool call;
- CPU time per agent step;
- bytes copied across host-device boundaries;
- queue and signal overhead;
- cancellation latency;
- failed or malformed tool-call rate;
- sensitivity to tool granularity and batch size.

9.3 PPA Modeling

After profiling the runtime, HATS can model candidate hardware primitives. The model should be conservative: area estimates for queue fabric, parser units, retrieval blocks, and sandbox engines should be compared against measured CPU energy, synchronization overhead, and data movement cost. The useful question is not whether HATS can beat a CPU on scalar control. The useful question is whether small local mechanisms can avoid enough boundary cost to justify their area.

10 Expected Contributions

HATS aims to contribute:

1. a scoped definition of hardware-friendly agentic tool primitives;
2. an Agent Tool Graph IR for tool-producing model workflows;
3. a HATS packet and runtime model inspired by HSA queues and signals;
4. a placement and lowering strategy for CPU, GPU, retrieval, parser, DMA, and sandbox targets;
5. a benchmark suite for agentic prefill and outer-loop overhead;
6. a PPA-driven method for deciding which hardware primitives are worth adding to future GPUs or heterogeneous packages.

11 Limitations

HATS is deliberately narrow. It does not claim that arbitrary tools belong on the GPU. It does not claim that accelerator hardware should own high-level agent planning. It does not solve distributed systems latency or make external APIs deterministic. It also does not assume that every agent workflow benefits from local tool acceleration.

The HATS hypothesis is strongest when:

- model state, indexes, and tool buffers are already local or resident;
- tool calls are frequent and fine-grained;
- continuation latency matters;
- agentic re-prefill events are frequent;
- structured validation sits in the prefill/decode loop;
- cancellation and dependency scheduling are hot paths.

The hypothesis is weakest when tool latency is dominated by remote services, human interaction, OS side effects, or broad scalar planning.

12 Conclusion

Agentic AI exposes a gap between dense model execution and the heterogeneous tool workflows wrapped around it. Current systems often run the model on a GPU and the agentic outer loop on a CPU, paying synchronization and data movement costs at every boundary. While many accelerators specialize the decode path, HATS specializes the agentic prefill path: the repeated construction, validation, retrieval, filtering, and ingestion of context before model continuation. HATS proposes a narrow alternative: keep the CPU in charge of policy and external side effects, but provide a coherent GPU-adjacent substrate for bounded local tool primitives.

The project should begin as a compiler and runtime system. That path makes the hardware story empirical. By representing tool workflows as graphs, lowering them to queues and signals, and profiling execution on existing systems, HATS can identify which primitives deserve hardware support. The eventual GPU is not the starting assumption. It is the consequence of measured agentic prefill bottlenecks.

References

- [1] HSA Foundation. *HSA Platform System Architecture Specification*. Version 1.2, 2018.
- [2] HSA Foundation. *HSA Programmer’s Reference Manual: HSAIL Virtual ISA and Programming Model, Compiler Writer, and Object Format (BRIG)*. Version 1.2, 2018.
- [3] HSA Foundation. *HSA Runtime Programmer’s Reference Manual*. Version 1.2, 2018.
- [4] HSA Foundation. *Towards a Formalization of the HSA Memory Model in the cat Language*. Version 1.2, 2018.
- [5] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *CGO*, 2021.
- [6] Khronos Group. *SPIR-V Specification*.
- [7] World Wide Web Consortium. *WebAssembly Core Specification*.
- [8] Linux Kernel Documentation. *eBPF Documentation*.